MÓDULO 1: INTRODUCCIÓN AL MACHINE LEARNING

INTRODUCCIÓN AL ML.

CONCEPTOS BÁSICOS DEL ML.

Comprender qué es el ML.

El machine learning (ML) es un tipo de inteligencia artificial (IA) que permite comprender y crear métodos que hagan posible que las máquinas *aprendan*. Estos métodos utilizan datos para mejorar el rendimiento de las computadoras en un conjunto de tareas.

Los científicos consideran al ML como un subcampo amplio de la IA. Los algoritmos de ML crean un modelo basado en datos de muestra, conocidos como datos de entrenamiento, para hacer predicciones o tomar decisiones sin estar programados explícitamente para hacerlo.

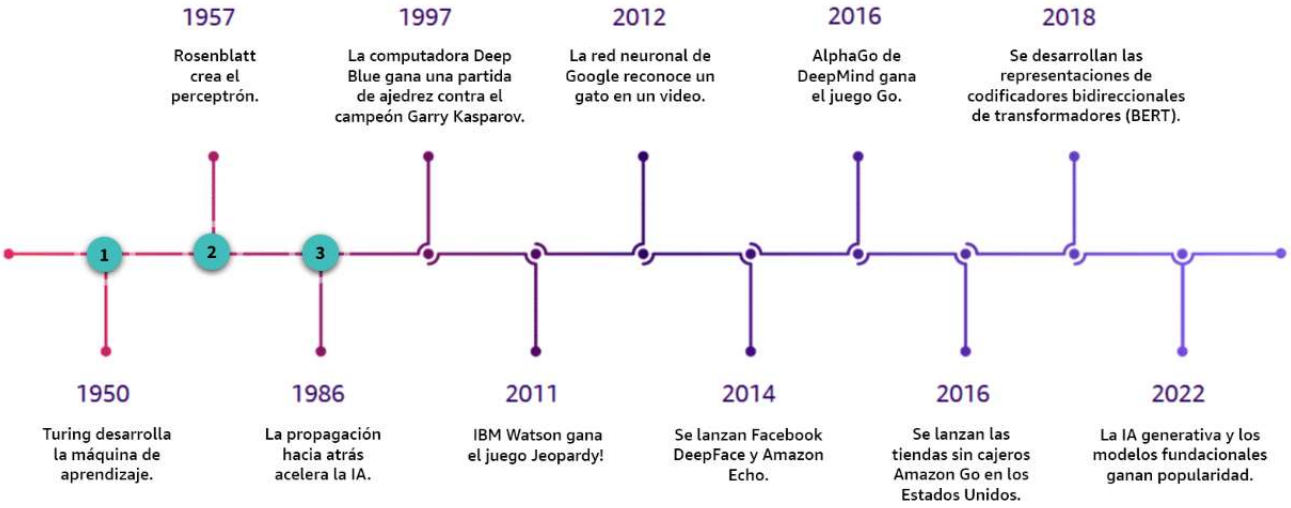
Una amplia variedad de aplicaciones utilizan algoritmos de ML cuando resulta difícil o inviable desarrollar algoritmos convencionales para realizar las tareas necesarias. Estas aplicaciones incluyen el filtrado de correos electrónicos, el reconocimiento de voz y la visión artificial.



El filtrado de los correos electrónicos, el reconocimiento de voz y la visión artificial son ejemplos de aplicaciones que utilizan algoritmos de ML.

Cronología del ML.

Actualmente, está ocurriendo un cambio importante en la historia de la inteligencia artificial y el machine learning (IA/ML) y muchas personas creen que será la fuerza principal detrás de la cuarta revolución industrial. Eche un vistazo a algunos de los eventos clave en el campo de la IA y el ML desde la década de 1950.



1. **Turing desarrolla la máquina de aprendizaje.**

   Cada vez que se habla de la historia de la IA, es necesario mencionar una de las primeras figuras de la inteligencia informática, Alan Turing. En 1950, Turing creó la famosa prueba de Turing, diseñada para determinar si una computadora tenía inteligencia real. Para aprobar, la computadora tenía que engañar a alguien haciéndole creer que era un humano a través de conversaciones lingüísticas. Esta prueba estableció algunas de las filosofías fundamentales de la inteligencia artificial.

2. **Rosenblatt crea el perceptrón.**

   El campo de la investigación del machine learning fue testigo de muchos logros, como el perceptrón electrónico de Frank Rosenblatt.

3. **La propagación hacia atrás acelera la IA.**

   En 1986, tres investigadores, David Rumelhart, Geoffrey Hinton y Ron Williams, publicaron un artículo sobre una técnica llamada propagación hacia atrás, que aceleró la investigación de la IA.

Descripción accesible del cronograma:

Cronología del machine learning, que va desde 1950 hasta 2022. Los eventos incluyen:

1950: Turing desarrolla la máquina de aprendizaje.

1957: Rosenblatt crea el perceptrón.

1986: la propagación hacia atrás acelera la IA.

1997: la computadora Deep Blue gana una partida de ajedrez contra el campeón Garry Kasparov.

2011: IBM Watson gana el juego Jeopardy!

2012: la red neuronal de Google reconoce un gato en un video.

2014: se lanzan Facebook DeepFace y Amazon Echo.

2016: DeepMind gana Go.

2018: se lanzan las tiendas sin cajeros Amazon Go en los Estados Unidos.

2018: se desarrollan las representaciones de codificadores bidireccionales de transformadores (BERT).

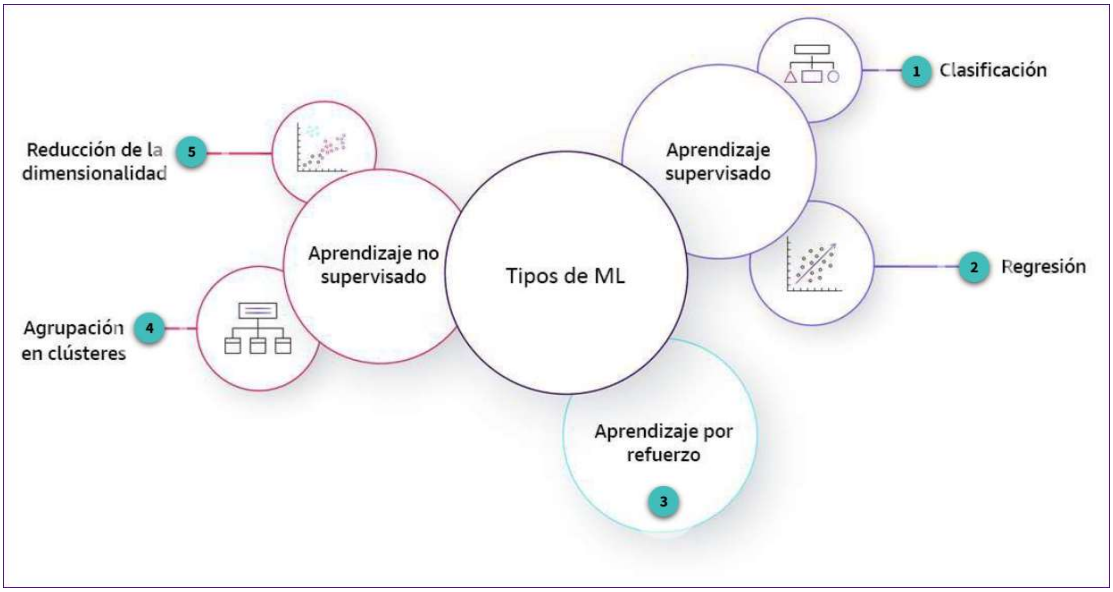
2022: la IA generativa y los modelos fundacionales ganan popularidad.

Tipos de ML.

Tradicionalmente, los enfoques de ML se dividen en tres categorías amplias: *aprendizaje supervisado*, *aprendizaje no supervisado* y *aprendizaje por refuerzo*. Corresponden a paradigmas de aprendizaje, según la naturaleza de la *señal* o la *retroalimentación* disponible para el sistema de aprendizaje.

En el siguiente diagrama, se incluye información general sobre los tipos de aprendizaje de ML. El aprendizaje supervisado se usa para hacer predicciones con datos etiquetados. Este tipo de aprendizaje incluye algoritmos de *clasificación* y *regresión*. El aprendizaje no supervisado se usa para encontrar patrones y estructuras en datos sin etiquetar, que tienen algoritmos para la *reducción de la dimensionalidad* y la *agrupación en clústeres*. El aprendizaje por refuerzo combina campos como las ciencias de la computación, la neurociencia y la psicología para determinar cómo asignar situaciones a acciones.

En este curso, profundizamos en el aprendizaje supervisado, que es la categoría de ML que se utiliza en Amazon SageMaker Canvas para crear modelos personalizados basados en los datos etiquetados del usuario.



1. **Ejemplos de clasificación:**
   - ✓ Detección de fraude de identidad
   - ✓ Clasificación de imágenes
   - ✓ Retención de clientes
   - ✓ Diagnósticos

2. **Ejemplos de regresión:**
   - ✓ Predicción de popularidad publicitaria
   - ✓ Previsiones meteorológicas
   - ✓ Previsión del mercado
   - ✓ Estimación de la esperanza de vida
   - ✓ Predicción del crecimiento demográfico

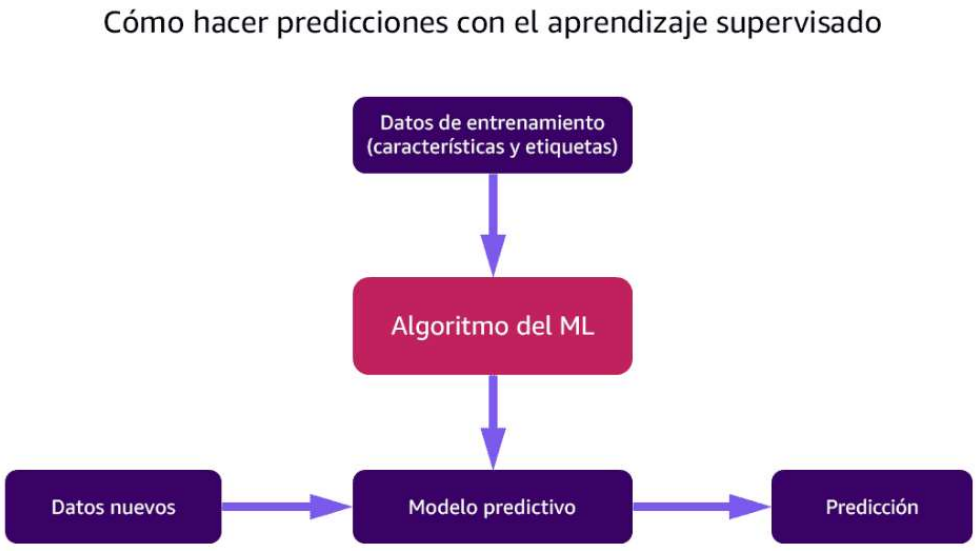
3. **Ejemplos de aprendizaje de refuerzo:**
   - ✓ Decisiones en tiempo real
   - ✓ Navegación robotizada
   - ✓ Tareas de aprendizaje
   - ✓ Adquisición de habilidades
   - ✓ IA en los videojuegos

4. **Ejemplos de agrupación de clústeres:**
   - ✓ Segmentación de clientes
   - ✓ Marketing orientado
   - ✓ Sistemas de recomendación

5. **Ejemplo de reducción de la dimensionalidad:**
   - ✓ Visualización de macrodatos
   - ✓ Compresión significativa
   - ✓ Descubrimiento de estructuras
   - ✓ Selección de características

Aprendizaje supervisado:

Un *profesor* presenta el modelo de aprendizaje supervisado con un conjunto de datos de ejemplos de entradas y sus salidas esperadas. Una entrada de ejemplo se denomina *característica* y la salida esperada *etiqueta*. El objetivo de entrenar un modelo de aprendizaje supervisado es que el modelo aprenda una regla general que asigne características a etiquetas.



El proceso muestra que los datos de entrenamiento (con sus características y etiquetas) ingresan al algoritmo de ML. El resultado es el modelo predictivo. Además, los datos nuevos ingresan al modelo predictivo y generan una predicción. El aprendizaje supervisado utiliza datos de entrenamiento para enseñar a los modelos a generar la salida esperada. Los datos de entrenamiento incluyen entradas y salidas correctas que ayudan al modelo a aprender.

Ejemplos de problemas de aprendizaje supervisado:

- **Regresión:**

  En la *regresión*, el problema es predecir un valor continuo. Puede utilizar diferentes métodos para comprender la relación entre las características (variables independientes) y un resultado (variable dependiente). Entre estos métodos se incluyen los siguientes:

  - ✓ Las **regresiones lineales** se utilizan para predecir el valor de una variable dependiente, dado el valor de una variable independiente. Utiliza una línea recta ajustada a un gráfico de puntos de datos.
  - ✓ Los **árboles de decisión** se utilizan como un modelo predictivo para sacar conclusiones de un conjunto de observaciones.
  - ✓ Los **bosques aleatorios** son conjuntos grandes de árboles de decisión. Para las tareas de regresión, se devuelve la predicción promedio de los árboles individuales.
  - ✓ Las **regresiones de redes neuronales** son una técnica en la que se utiliza una red neuronal artificial para modelar y predecir valores numéricos continuos.

- **Clasificación:**

  En la *clasificación*, el problema consiste en predecir si un elemento pertenece a una categoría o clase en particular. Puede utilizar diferentes métodos de modelado predictivo en los que la etiqueta de clase esté prevista para un ejemplo específico de datos de entrada. Entre estos métodos se incluyen los siguientes:

  - ✓ Las **regresiones logísticas** se utilizan para estimar la probabilidad de que ocurra un evento en función de un conjunto de datos determinado de variables independientes. La predicción suele tener resultados binarios, como 1 o 0, sí o no, verdadero o falso, positivo o negativo.
  - ✓ Las **máquinas de vector de soporte** son modelos de aprendizaje supervisado con algoritmos de aprendizaje asociados que analizan los datos para el análisis de clasificación y regresión.
  - ✓ **Bayesiano ingenuo** es un algoritmo basado en el teorema de Bayes, llamado así por Thomas Bayes, que describe la probabilidad de que ocurra un evento. Hace la suposición ingenua de la independencia condicional entre cada par de características dado el valor de la variable de clase.

  También puede usar árboles de decisión, bosques aleatorios y redes neuronales, que tienen la misma definición de agrupación en clústeres que de regresión.

Casos prácticos de ML.

¿Tiene en mente un resultado empresarial específico? Comience con uno de los siguientes casos prácticos para acelerar su recorrido hacia el ML.



- **Procesamiento de lenguaje natural:**

  Casos prácticos del procesamiento de lenguaje natural (NLP):

  - ✓ Minería de análisis empresariales y de centros de llamadas
  - ✓ Indexación y búsquedas en reseñas de productos
  - ✓ Gestión de reportes legales
  - ✓ Procesamiento de documentos financieros

- **Análisis de imágenes y videos:**

  Casos prácticos del análisis de imágenes y videos:

  - ✓ Detección de contenido inapropiado
  - ✓ Verificación de identidad en línea
  - ✓ Optimización del análisis multimedia
  - ✓ Envío de alertas a hogares inteligentes conectados

- **Servicios financieros:**

  Casos prácticos de servicios financieros:

  - ✓ Análisis y regulación de riesgos
  - ✓ Segmentación de clientes
  - ✓ Venta cruzada y venta adicional

- ✓ Gestión de campañas de ventas y marketing
- ✓ Evaluación de la capacidad crediticia

- **Sanidad y ciencias de la vida:**

  Casos prácticos de sanidad y ciencias de la vida:

  - ✓ Alertas y diagnósticos a partir de datos de pacientes en tiempo real
  - ✓ Identificación de enfermedades y estratificación de riesgos
  - ✓ Optimización de la evaluación de los pacientes
  - ✓ Gestión proactiva de la salud
  - ✓ Análisis de sentimiento de los proveedores de sanidad

- **Fabricación:**

  Casos prácticos de fabricación:

  - ✓ Mantenimiento predictivo o supervisión de condiciones
  - ✓ Estimación de reserva de garantía
  - ✓ Propensión a la compra
  - ✓ Previsión de la demanda
  - ✓ Optimización de procesos

- **Sistemas de vehículos autónomos:**

  Casos prácticos de sistemas de vehículos autónomos:

  - ✓ Sistemas avanzados de asistencia a la conducción (ADAS)
  - ✓ Conducción autónoma
  - ✓ Experiencias de entretenimiento a bordo del vehículo

- **Sistemas de recomendaciones:**

  Casos prácticos de sistemas de recomendaciones:

  - ✓ Optimización de las recomendaciones
  - ✓ Segmentación de clientes con mayor exactitud
  - ✓ Maximización del valor de los datos
  - ✓ Promoción de artículos mediante reglas empresariales

- **Comercio minorista:**

  Casos prácticos de comercio minorista:

  - ✓ Planificación predictiva del inventario
  - ✓ Venta adicional y marketing multicanal
  - ✓ Focalización y segmentación de mercados
  - ✓ Rendimiento de la inversión (ROI) del cliente y valor del ciclo de vida

- **Viajes y hostelería:**

  Casos prácticos de viajes y hostelería:

  - ✓ Programación de aeronaves
  - ✓ Precios dinámicos
  - ✓ Comentarios e interacciones de los clientes en las redes sociales
  - ✓ Resolución de reclamos de clientes
  - ✓ Patrones de tráfico y gestión de embotellamientos

- **Energía:**

  Casos prácticos de energía:

  - ✓ Análisis del uso de energía
  - ✓ Procesamiento de datos sísmicos
  - ✓ Emisiones y comercio de carbono
  - ✓ Precios específicos para cada cliente
  - ✓ Gestión de la "red inteligente"
  - ✓ Optimización de la demanda y el suministro de energía

TIPOS DE PROBLEMAS QUE SE PUEDEN RESOLVER CON ML

Evaluación de casos empresariales.

Si su organización está pensando en utilizar ML, usted debe tener claro cuál es el problema empresarial que se necesita resolver y qué beneficios se pueden obtener. Debe poder evaluar el valor empresarial en relación con ciertos objetivos comerciales y factores de éxito. Esto es válido para cualquier solución técnica, pero cuando se piensa en soluciones de ML, esta etapa es particularmente difícil. Esto se debe a que el ML es una tecnología en continuo crecimiento.

Después de decidir cuáles son sus criterios de éxito, evalúe la capacidad de su organización para lograr ese objetivo. El objetivo debe ser factible y allanar el camino a la producción. Incluya a todas las partes relevantes desde el principio para garantizar su alineamiento con el objetivo. Identifique cualquier procedimiento empresarial nuevo que pueda surgir de la iniciativa.



1. **Determinar si el ML es conveniente:**

   Para iniciar la revisión, determine si el ML es el enfoque apropiado para alcanzar el objetivo empresarial.
2. **Evaluar las opciones para lograr los objetivos:**

   Evalúe todas las opciones disponibles para lograr el objetivo.
3. **Considerar la exactitud, el costo y la escabilidad:**

   Determine qué tan exactos serían los resultados teniendo en cuenta el costo y la escalabilidad de cada enfoque.
4. **Calificar los datos de entrenamiento:**

   Para que un enfoque basado en ML funcione correctamente, debe asegurarse de que el algoritmo disponga de suficientes datos de entrenamiento relevantes y de alta calidad.
5. **Evaluar los orígenes de datos disponibles:**

   Evalúe cuidadosamente los datos para asegurarse de que los orígenes de datos correctos estén disponibles y sean accesibles.

A continuación, se presentan los diferentes pasos que debe tener en cuenta en la revisión empresarial del ML:

- Entender los requisitos empresariales
- Formular una pregunta empresarial
- Revisar la viabilidad del ML y los requisitos de datos de un proyecto
- Evaluar el costo de la adquisición, el entrenamiento, la inferencia y las predicciones incorrectas de los datos

- Revisar trabajos comprobados o publicados en dominios similares, si están disponibles
- Determinar las métricas clave de rendimiento, incluidos los errores aceptables
- Definir la tarea de machine learning en función de la pregunta empresarial
- Identificar las características clave e imprescindibles
- Diseñar pruebas de concepto (POC) pequeñas y específicas para validar todos los elementos anteriores de esta lista
- Evaluar si la incorporación de orígenes de datos externos mejorará el rendimiento de los modelos
- Establecer rutas hacia la producción
- Considerar nuevos procesos empresariales que podrían surgir de la implementación
- Alinear las partes interesadas relevantes con la iniciativa

Cuándo usar ML.

Es importante recordar que el ML no es una solución para todos los tipos de problemas. En determinados casos, puede desarrollar soluciones sólidas sin utilizar técnicas de ML. Por ejemplo, no necesita ML si puede determinar un valor objetivo mediante reglas simples, cálculos o pasos predeterminados. Estos pasos se pueden programar sin necesidad de recurrir a un aprendizaje basado en datos. En algunas situaciones, el ML es una buena opción.

Situaciones viables para la implementación de ML:

- **No se pueden codificar las reglas:** muchas tareas humanas, como reconocer si un correo electrónico es spam o no, no se pueden resolver adecuadamente con una solución simple (determinista) basada en reglas. Una gran cantidad de factores pueden influir en la respuesta. Cuando las reglas dependen de demasiados factores, se superponen o es necesario ajustarlas con mucha exactitud, resulta difícil para un humano codificarlas con precisión. Puede utilizar ML para resolver este problema de forma eficaz.
- **No se puede escalar:** es posible reconocer manualmente unos cientos de correos electrónicos y decidir si son spam o no. Sin embargo, esta tarea se vuelve tediosa cuando hay millones de correos electrónicos. Las soluciones de ML son eficaces para gestionar problemas a gran escala.

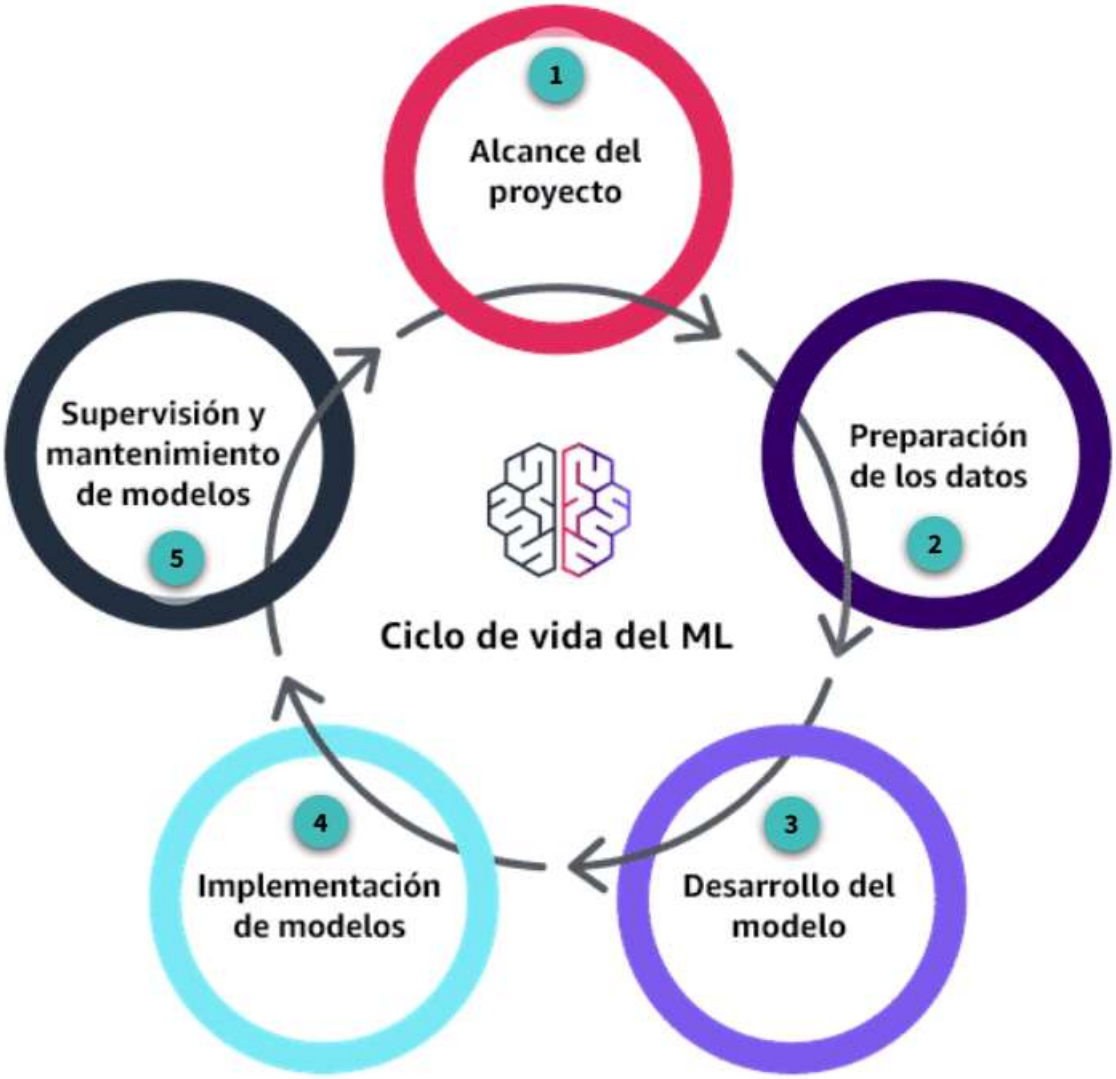
CICLO DE VIDA DEL ML

La creación y el funcionamiento de una carga de trabajo de ML típica es un proceso iterativo que consta de varias fases. El proceso de ML incluye las siguientes etapas:

- Identificación de objetivos empresariales
- Planteamiento del problema de ML

- Recopilación de datos
- Integración y preparación de datos
- Ingeniería de características
- Entrenamiento de modelos
- Validación de modelos
- Evaluación empresarial
- Implementación en producción (implementación de modelos e inferencia de modelos)

En el siguiente diagrama, se describe el proceso del ciclo de vida del ML, junto con los expertos en el tema (SME) y las partes interesadas empresariales que participan en las diferentes fases del proceso.



1. **Alcance del proyecto:**

   Use el alcance del proyecto para realizar lo siguiente:

   - ✓ Definir el caso práctico empresarial
   - ✓ Definir el marco de trabajo de los problemas de ML
   - ✓ Involucrar a los propietarios de empresas y a los expertos en dominios

2. **Preparación de los datos:**

   Use la preparación de datos para realizar lo siguiente:

   - ✓ Ingesta de datos
   - ✓ Exploración y limpieza de datos
   - ✓ Ingeniería y selección de características
   - ✓ Involucramiento de ingenieros y científicos de datos

3. **Desarrollo de modelos:**

   Use el desarrollo de modelos para realizar lo siguiente:

   - ✓ Experimentación y entrenamiento de modelos
   - ✓ Prueba de modelos
   - ✓ Evaluación de modelos
   - ✓ Involucramiento de científicos de datos

4. **Implementación de modelos:**

   Use la implementación de modelos para realizar lo siguiente:

   - ✓ Integración y entrega continuas (CI/CD)
   - ✓ Control de versiones
   - ✓ Predicción por lotes y en tiempo real
   - ✓ Involucramiento de científicos de datos y operaciones de desarrollo (DevOps)

5. **Supervisión y mantenimiento de modelos:**

   Use la supervisión y el mantenimiento de modelos para realizar lo siguiente:

   - ✓ Detección de desviaciones
   - ✓ Definición del objetivo de retención
   - ✓ Alertas y registro
   - ✓ Involucramiento de científicos de datos y DevOps

Fases del ciclo de vida.

En esta sección, se presenta información general de alto nivel sobre las distintas fases del ciclo de vida del ML. Ayuda a delimitar el debate sobre la seguridad, el cumplimiento y la operacionalización de las prácticas recomendadas de ML de Amazon Web Services (AWS).

- **Alcance del proyecto: definir el problema y los objetivos empresariales.**

  Los expertos en dominios y los propietarios de empresas son los que más participan en esta fase, en la que se definen las métricas de éxito. La identificación de los indicadores de rendimiento claves (KPI) y la determinación de los requisitos normativos y de cumplimiento también se incluyen en esta fase. Los científicos de datos suelen trabajar con los SME para delimitar el problema empresarial de manera que puedan utilizarlo para desarrollar una solución de ML viable.

- **Preparación de datos: recopilar y preparar todos los datos relevantes.**

  Esta fase implica recopilar y preparar todos los datos relevantes de varios orígenes de datos. Los ingenieros de datos con experiencia en el uso de técnicas de macrodatos para la extracción, transformación y carga (ETL) de los datos suelen ocupar este puesto.

  Para cumplir los requisitos de auditoría y cumplimiento, es fundamental asegurarse de que los datos estén versionados y de que se registre su linaje. Los científicos de datos exploran los datos, identifican las características de entrada y las variables de objetivo, realizan análisis de valores atípicos e implementan las transformaciones de datos necesarias una vez que obtienen acceso a los conjuntos de datos sin procesar. También es clave garantizar que cualquier cambio que los científicos de datos realicen en los datos de entrenamiento se pueda utilizar en la producción para hacer inferencias.

- **Desarrollo de modelos: desarrollar y evaluar modelos.**

  La siguiente es la fase de desarrollo y evaluación de modelos. Los científicos de datos determinan el marco de trabajo que se va a utilizar. También definen los conjuntos de datos que quedan fuera de la muestra y los que se consideran obsoletos de la siguiente manera:

  - ✓ Los datos que quedan *fuera de la muestra* son aquellos que no se utilizaron durante el proceso de desarrollo y optimización. Estos datos se utilizan para evaluar el rendimiento de un modelo.
  - ✓ Los *datos obsoletos provienen de un período* de tiempo completamente diferente al que se utilizó para el desarrollo de modelos.

Los científicos de datos también experimentan de las siguientes maneras:

- ✓ Uso de diferentes algoritmos de ML
- ✓ Variación de un conjunto de parámetros denominados hiperparámetros para controlar el rendimiento y la exactitud de los modelos. La velocidad de aprendizaje es un ejemplo de hiperparámetro. Se trata de la velocidad a la que un algoritmo actualiza las ponderaciones en una red neuronal. Una velocidad demasiado alta impedirá alcanzar un valor óptimo, mientras que una velocidad demasiado baja ralentizará el entrenamiento de los modelos.
- ✓ Incorporación de datos de entrenamiento adicionales (en algunos casos)

- **Implementación de modelos: ejecute modelos entrenados con conjuntos de datos.**

  En esta fase, se toman los modelos entrenados y se los ejecuta con conjuntos de datos obsoletos y que quedaron fuera de la muestra. Luego, se elige el modelo o los modelos que arrojan los mejores resultados y se acercan más a las métricas determinadas en la fase 1. Debe versionar correctamente los artefactos del modelo y cualquier código correspondiente y almacenarlo en un repositorio de código centralizado o en un sistema de administración de artefactos.

  Esta etapa del proceso es experimental. Los científicos de datos pueden volver a la etapa de recopilación de datos o ingeniería de características si el rendimiento del modelo es sistemáticamente deficiente. Los científicos de datos también deben dar motivos del rendimiento deficiente o explicar la influencia de una característica o un modelo en las predicciones. La interpretabilidad de los modelos se analiza más adelante en el curso.

- **Supervisión y mantenimiento de modelos: implementar los modelos en producción.**

  A menudo, esta fase es la de mayor impacto y dificultad. Existe una brecha entre las tecnologías y las competencias que se utilizan para crear e implementar modelos en producción. Una gran parte del éxito de esta fase depende de la colaboración estrecha entre los profesionales de infraestructura durante el proceso de la toma de decisiones. Estos profesionales incluyen ingenieros de DevOps, científicos de datos, ingenieros de datos, expertos en dominios, usuarios finales y propietarios de empresas. Debe haber métricas estandarizadas y todos los responsables de la toma de decisiones deben poder interpretarlas de manera directa.

  En la mayoría de las organizaciones, los ciclos de vida del ML finalizan con la fase de implementación. Las organizaciones deben tener una validación paralela, en la que los modelos se implementan, pero no se

integran en el flujo de trabajo de producción para detectar las diferencias entre los datos de entrenamiento y los datos en vivo. De esta manera, se garantiza que el modelo siga funcionando según lo esperado cuando reciba datos de los sistemas de producción. Una vez que esta validación funcione, puede usar las predicciones del modelo en los flujos de trabajo de producción.

Sin embargo, para que los modelos de ML sean efectivos a largo plazo, es necesario supervisar continuamente el modelo en tiempo real (si es posible). Esto ayuda a determinar su rendimiento, ya que la exactitud de los modelos puede disminuir con el tiempo. Es posible que el modelo pierda exactitud debido a los cambios en las propiedades estadísticas de las características de entrada, las variables objetivo, las relaciones entre las variables, los patrones cambiantes de los clientes, las tendencias, etc. Si el rendimiento del modelo cae por debajo de un determinado umbral, es posible que deba volver a entrenar e implementar el modelo.

Servicios de ML con poco código y sin código.

El entrenamiento de modelos de ML y la preparación de datos pueden ser complejos y los servicios de código tradicionales requieren experiencia en codificación. Sin embargo, los servicios con poco código y sin código hacen que el ML sea más accesible para los usuarios con experiencia limitada en codificación.

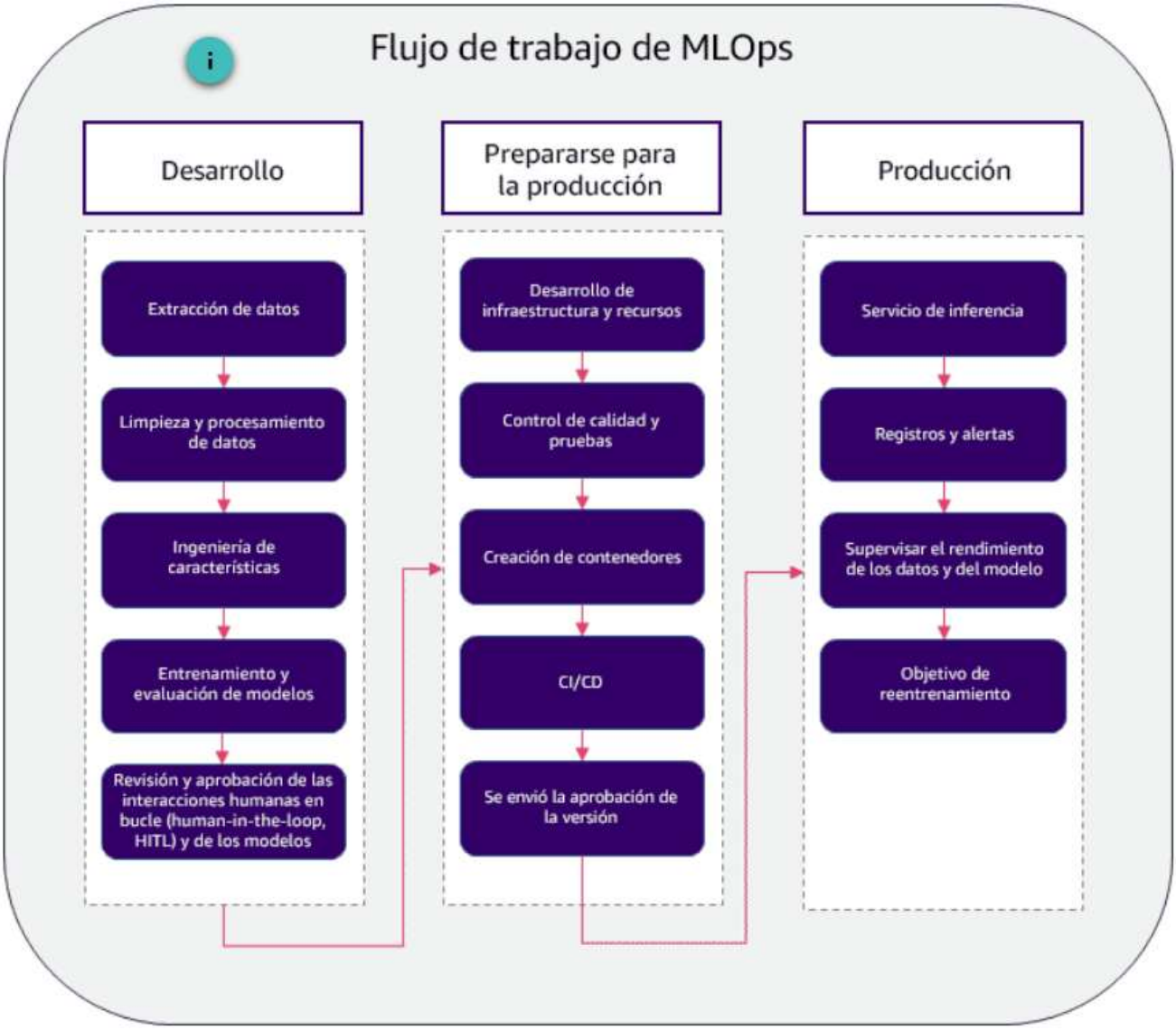
	Servicios sin código	**Servicios de bajo código**	**Servicios de código tradicionales**
Competencia en programación	Usuarios con mínima competencia en programación	Usuarios con competencia intermedia en programación que pueden personalizar scripts y fragmentos generados	Programadores competentes que pueden crear scripts desde cero
Estilo de interfaz de usuario	✓ Arrastrar y soltar. ✓ Componentes prediseñados ✓ Machine learning automatizado (AutoML) ✓	Combina elementos visuales con lenguajes de scripting o fragmentos de código.	Entorno de desarrollo integrado (IDE)

Casos prácticos	✓ Proyectos sencillos. ✓ Experimentación rápida. ✓ Entrega rápida de proyectos.	Proyectos más complejos.	✓ Proyectos de alta complejidad. ✓ Requiere control sobre el entorno de ML.
Beneficios	✓ ML accesible para usuarios no técnicos. ✓ Promueve la innovación y una adopción más amplia del ML. ✓ Procesos automáticos que reducen la duración del desarrollo. ✓ El usuario puede centrarse únicamente en el problema empresarial.	Más flexible que los servicios sin código, ya que se permite la personalización por parte del usuario	La variedad de modelos, marcos de trabajo y opciones de entorno de ML más amplia

Los servicios de ML con poco código y sin código son herramientas poderosas que amplían el acceso a la tecnología transformadora del ML.

Operaciones y flujos de trabajo de ML.

El ciclo de vida del ML elimina los silos entre las diferentes partes interesadas que deben participar en los proyectos de ML para plasmar el valor empresarial. La disolución de los silos comienza con las actividades de adquisición y modelado de datos del equipo de ciencia de datos. El equipo debe tener una comprensión clara de los objetivos empresariales de la aplicación de ML y de los problemas de gobernanza y cumplimiento que deben tenerse en cuenta.



Flujo de trabajo de MLOps con las partes interesadas de ML.

Información sobre este gráfico:

Hay tres grupos de partes interesadas. El primer grupo es el de desarrollo. Sus tareas incluyen la extracción de datos, la limpieza y el procesamiento de datos, la ingeniería de características, el entrenamiento y la evaluación de modelos, y la revisión y aprobación de modelos. El segundo grupo se encarga de los preparativos para la producción. Sus tareas incluyen el desarrollo de infraestructura y recursos, el control de calidad y las pruebas, la creación de contenedores, la CI/CD y el envío para la aprobación de lanzamiento. El tercer grupo se encarga de la producción. Sus tareas incluyen el servicio de inferencias, el registro y las alertas, la supervisión del rendimiento de los datos y los modelos, y los objetivos de la repetición de entrenamientos.

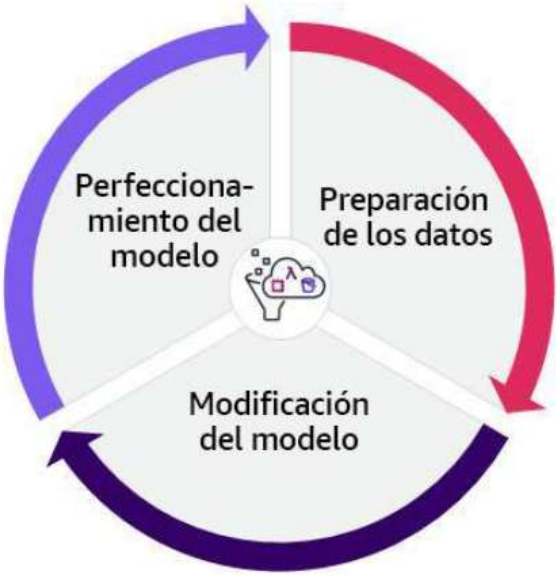
MLOps es la respuesta para la automatización de los flujos de trabajo de ML. Incorporar MLOps es como incorporar DevOps; es necesario asimilar una mentalidad y una forma de trabajar diferentes. Sin embargo, el ROI que genera este tipo de iniciativa vale la pena.

DESAFÍOS RELAICONADOS CON EL PROCESAMIENTO Y LA OBTENCIÓN DE INFORMACIÓN A PARTIR DE DATOS

Datos del proyecto de ML.

Un modelo de ML es tan bueno como los datos utilizados para entrenarlo. Cuando se usan datos de mala calidad, se suele decir que *basura entra, basura sale*. Una vez recopilados los datos, los siguientes pasos son integrar, preparar y anotar los datos. Si bien la preparación y el procesamiento de datos suele ser un proceso rutinario, podría decirse que es la etapa más importante para determinar si un modelo funcionará según lo previsto o tendrá un rendimiento deficiente.

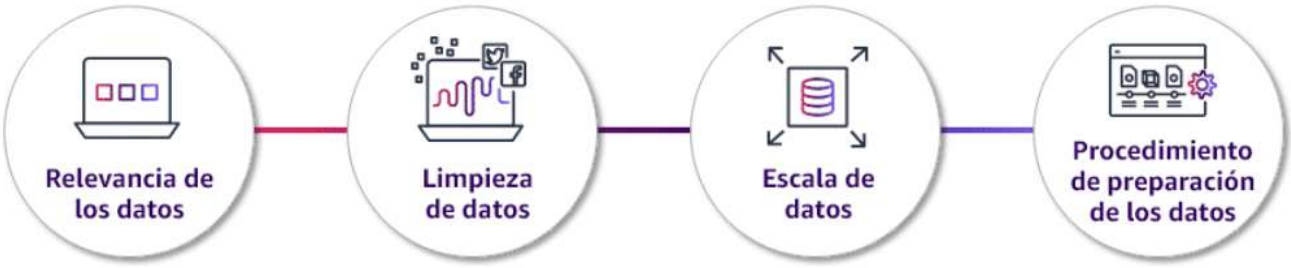
La mayoría de las dificultades de la ciencia de datos surgen durante la fase de preparación de los datos. Aunque se cree que la creación de un proyecto de aprendizaje automático completo es un proceso secuencial, en la práctica, con frecuencia es cíclico. Para lograr el rendimiento esperado, se intercambia entre la preparación de datos, la modificación del modelo y el perfeccionamiento del modelo.



Un proyecto de ML consiste en un proceso iterativo de preparación de datos, modificación del modelo y perfeccionamiento del modelo.

Desafíos de la preparación de datos.

Debido al volumen de datos utilizado, uno de los mayores obstáculos en el análisis de macrodatos y ML es la etapa de preparación de los datos. Los desafíos incluyen la relevancia, la limpieza, la escala y el procedimiento de preparación de los datos. En esta sección, se proporcionan detalles sobre estos desafíos.



Desafíos de la preparación de datos.

Relevancia de los datos:

El primer grupo de desafíos está vinculado con el origen de los datos. Contar con los datos adecuados es fundamental para describir el problema con suficiente detalle para el proyecto. Uno de los desafíos clave del análisis de macrodatos es elegir el subconjunto correcto que describa el problema en cuestión. A menudo, los problemas de relevancia de los datos pueden incluso requerir el reemplazo del conjunto de datos.

- **Datos sesgados:**

  El sesgo en los datos es un error que se produce cuando ciertos elementos de un conjunto de datos están sobreponderados o sobrerrepresentados. Los conjuntos de datos sesgados no representan con exactitud el caso práctico de un modelo de ML, lo que conduce a resultados sesgados, prejuicios sistemáticos y baja precisión.

  Considere un conjunto de datos que incluya las preferencias de los clientes en función del género. Tener un número desigual de entradas hará que la mayoría de los modelos de ML estén sesgados. Esto puede generar que los modelos, por ejemplo, prefieran un grupo sobre otro.

- **Efectos estacionales en los datos:**

  Hay muchos ejemplos de datos con aparentes efectos estacionales. Por ejemplo, el comportamiento de los clientes o los datos financieros suele tener tendencias. Este desafío requiere una cuidadosa selección de subconjuntos o la implementación del procedimiento de eliminación de tendencias, que excluye del conjunto de datos los valores que causan efectos estacionales.

- **Los datos cambian a lo largo del tiempo:**

  Por lo general, las observaciones cambian con el tiempo. Por ejemplo, los registros del comportamiento de los consumidores de una década atrás no son útiles en la actualidad. Además, los datos históricos a menudo pueden ser engañosos porque describen procesos que no son similares a los actuales. Esta propiedad de los datos puede requerir el reemplazo del conjunto de datos.

- **Relevancia de los subconjuntos de entrenamiento y prueba:**

  Al igual que ocurre con los datos de entrenamiento sesgados, el uso de datos de prueba que solo consideren una parte de todos los eventos y características posibles puede arrojar resultados inexactos. Es probable que los modelos de ML entrenados con conjuntos de datos sesgados que son estadísticamente incorrectos provoquen la introducción del modelo con un error. Por lo tanto, es fundamental tener datos de prueba y entrenamiento completos que no estén sesgados.

  Una forma de evitar estos problemas es utilizar varios conjuntos de pruebas diferentes. Otra manera es seleccionar uno del conjunto de datos principal como submuestra que contenga la misma información y características en una cantidad menor.

Limpieza de datos:

Existe una amplia variedad de problemas diferentes que pueden aparecer en un conjunto de datos. Por ejemplo, corregir los formatos incorrectos es uno de los desafíos más importantes de la ciencia de datos. Existen varios métodos para tratar estos problemas.

- **Valores atípicos:**

  Los valores atípicos de los datos son registros específicos que se desvían radicalmente de la media. Estos valores pueden reflejar observaciones inexactas de un evento genuino poco frecuente o sorpresas que ocurren de manera natural en los datos. En cualquier caso, los valores atípicos no ayudan al modelo de ML en su conjunto. Algunos modelos de ML, como los bosques aleatorios, ignorarán los valores atípicos. Es posible que otros, como las regresiones lineales, no los tengan en cuenta.

  Sin embargo, los valores atípicos tienen el potencial de reducir radicalmente la exactitud de la mayoría de los sistemas de ML. Dado que los valores atípicos pueden ser difíciles de detectar, la mayoría de las aplicaciones de ML no los incluyen.

- **Duplicados:**

  Si bien eliminar registros duplicados no es un desafío nuevo, omitirlo puede ser un error grave. Los valores duplicados pueden acentuar observaciones particulares, lo que mejora el funcionamiento de los algoritmos de ML para casos repetidos en el conjunto de datos de entrenamiento.

  Sin embargo, tener registros duplicados de datos de entrenamiento en el subconjunto de pruebas también puede resultar confuso para quienes validan los modelos de ML. Por lo tanto, los valores duplicados deben excluirse de los datos.

- **Valores faltantes:**

  A menos que se corrijan de alguna manera, los valores faltantes pueden reducir significativamente los resultados de un futuro modelo. Según el algoritmo, es posible compensar los datos faltantes de varias maneras. Puede eliminar los registros con datos faltantes, reemplazar los valores faltantes o aplicar un algoritmo especial (por ejemplo, reemplazar con media, modo y ceros). Este paso es fundamental y requiere un análisis adicional antes de tomar cualquier medida.

  Es clave saber cuántos valores faltan o si las características faltantes están correlacionadas con alguna otra cosa. Por ejemplo, si la falta de datos ocurre solo en algún tipo de evento especial. Estos aspectos pueden cambiar la decisión final y, en consecuencia, afectar al rendimiento del modelo.

- **Ruido en los datos:**

  La palabra *ruido* se usa comúnmente para describir valores numéricos, pero también puede describir texto o cualquier otro dato. Hay muchos tipos diferentes de ruido, como el ruido estático y el dinámico. Sin embargo, ninguno proporciona información valiosa al modelo y complican la etapa previa al procesamiento.

  El filtrado del ruido es un proceso complejo y arriesgado. Requiere un nivel de conocimiento y experiencia adecuado. Cuando se aplican técnicas de eliminación de ruido, también es posible que se eliminen datos valiosos.

Escala de datos:

Es preferible tener grandes cantidades de datos a volúmenes insuficientes. Por ejemplo, puede ser sencillo eliminar los valores atípicos y las variables faltantes cuando se necesitan más datos, o simplemente puede utilizar una muestra para el entrenamiento. Sin embargo, incluso después del preprocesamiento, con frecuencia, una

cantidad modesta de datos puede causar problemas. Como resultado, una de las principales dificultades del análisis de datos es elegir la cantidad adecuada de datos sin sobreajustar el modelo. El sobreajuste es un comportamiento de ML no deseado que se produce cuando el modelo de ML proporciona predicciones exactas para los datos de entrenamiento, pero no para los datos nuevos.

- **Volúmenes de datos:**

  Tener un conjunto de datos enorme generalmente requiere algunas técnicas especiales de almacenamiento de datos o computación en la nube para procesarlo. Sin embargo, los subconjuntos de gran tamaño pueden crear una infraestructura de desarrollo complicada y aumentar el tiempo de entrenamiento. El procesamiento de grandes volúmenes de datos o registros también puede llevar mucho tiempo. Cuando el volumen de datos es mayor al que el modelo puede manejar, una selección cuidadosa de la muestra o incluso un nuevo muestreo puede mejorar significativamente los resultados del modelo.

- **Subconjuntos de datos aleatorios:**

  La aplicación de técnicas adecuadas de división de datos es extremadamente importante cuando se trata de modelos de entrenamiento y prueba. A menudo, los ingenieros utilizan datos secuenciales. Estos grupos solo describen un período o grupo en particular. Esta modalidad de uso de los datos disminuye el rendimiento de cualquier modelo de ML. Cuando se estén aleatorizando subconjuntos de pruebas o entrenamiento, verifique que los científicos de datos usen técnicas de división de datos para evitar este problema.

- **Representación de datos:**

  A menudo, hay varias maneras de representar o almacenar datos. Por ejemplo, considere los *formatos de datos anchos y largos*. Ambos tienen sus ventajas e inconvenientes. Seleccionar el más adecuado puede mejorar significativamente el rendimiento durante el proceso de entrenamiento. Una solución que cubre posibles confusiones es el *principio de datos ordenados*. El principio de datos ordenados recomienda una forma estándar de organizar los valores de los datos en un conjunto de datos. Sin embargo, todavía no existe un método único.

- **Coincidencia de datos:**

  Con bastante frecuencia, la coincidencia de datos describe la eliminación de duplicados. Sin embargo, es un gran desafío cuando se trata de comparar varios conjuntos de datos de múltiples orígenes que tienen formatos diferentes. El formato de los datos puede variar según el origen, por lo que la eliminación de duplicados es un proceso de estandarización. En consecuencia, los registros similares no siempre tienen

que ser idénticos. Determinar la métrica de comparación, los estándares de datos adecuados, etc., es un proceso largo y complejo. Hacer coincidir varios conjuntos de datos de gran tamaño es uno de los principales desafíos del análisis de macrodatos.

Procedimiento de preparación de datos:

Durante la fase de preparación de datos, la mayoría de los problemas están relacionados con los datos en sí. Sin embargo, existen algunos desafíos importantes debido al proceso de preparación. Iniciar un procedimiento de preparación en producción no es una tarea sencilla. Por sí solo, el establecimiento de una canalización que funcione puede ser un desafío importante para el análisis de macrodatos.

- **Incluir datos nuevos en los procesos:**

  Muchos proyectos, incluidos los que se basan en ML, no tienen un tiempo fijo. Muy a menudo, los modelos se entrenan o prueban continuamente. Además, como los datos cambian de forma con el tiempo, es necesario desarrollar nuevas técnicas de preparación. En ocasiones, ajustar la canalización de procesamiento para un uso continuo puede requerir un esfuerzo significativo. Por lo tanto, es necesario mantener el modelo ajustado para los datos nuevos. Compruebe constantemente el rendimiento (y el cambio del rendimiento con el tiempo) y cree una canalización de entrenamiento continuo.

- **Automatización de los procesos de ETL y ELT:**

  El proceso de extracción, **transformación** y carga (ETL) utiliza un conjunto de reglas empresariales para procesar datos de varios orígenes antes de transferirlos a un almacén de datos central.

  El enfoque de extracción, **carga** y transformación (ELT) carga los datos en su forma actual y los transforma en una etapa posterior, según el caso práctico y los requisitos de análisis.

  A veces, primero es necesario recopilar y estandarizar los datos necesarios para que el modelo funcione. Por ejemplo, es posible que tenga un rastreador web en el que se agreguen nuevos registros al conjunto de datos de forma continua. Entre los desafíos se incluyen los siguientes:

  - ✓ En primer lugar, la fase de extracción se complica.
  - ✓ En segundo lugar, la transformación de los datos continuos puede ser un desafío importante.
  - ✓ En tercer lugar, puede ser imposible evaluar correctamente la distribución de un valor determinado.

Además, como ya se explicó, ingresar entradas que se ajusten al modelo puede ser una tarea compleja. Siempre que haya un elemento de uso continuo o repetido de datos, la técnica de preparación de datos se convierte en una parte integral de todo el proyecto. Requiere una implementación especial, cálculos rápidos, cómputos optimizados, etc.

La herramienta de machine learning con poco código y sin código simplifica el procesamiento de datos:

Los servicios con poco código y sin código ofrecen características para simplificar y automatizar el procesamiento de datos. De esta manera, el preprocesamiento de datos se vuelve accesible para usuarios con diferentes niveles técnicos.

El preprocesamiento de datos sin código proporciona interfaces fáciles de usar con funciones prediseñadas para tareas comunes.



1. **Datos faltantes:**
   Completar espacios vacíos con promedios o medianas para imputar valores faltantes.
2. **Valores atípicos:**
   Detectar y eliminar puntos de datos atípicos que podrían sesgar los resultados.
3. **Conversión de datos:**
   Convertir tipos de datos, como valores de texto en valores numéricos, para lograr la compatibilidad del modelo.
4. **Limpieza de textos:**
   Eliminar palabras de detención, signos de puntuación y caracteres especiales para limpiar el texto.
5. **Ajustes de escala de datos:**
   Normalizar y ajustar la escala de los datos.

Los servicios sin código aceleran el preprocesamiento de los datos. No se requieren conocimientos de programación y, a menudo, se guía mediante asistentes, instrucciones paso a paso o solicitudes de chat en lenguaje natural.

CREACIÓN Y EVALUACIÓN DE MODELOS DE ML.

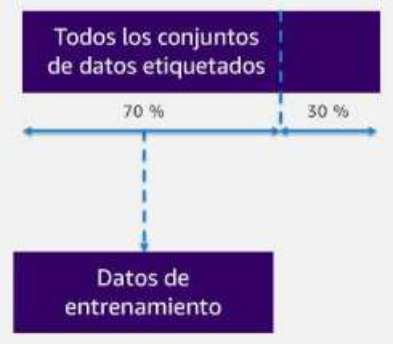
INTRODUCCIÓN A LA CREACIÓN DE MODELOS.

Creación de un modelo.

La creación de aplicaciones de ML es un proceso iterativo que incluye una secuencia de pasos.

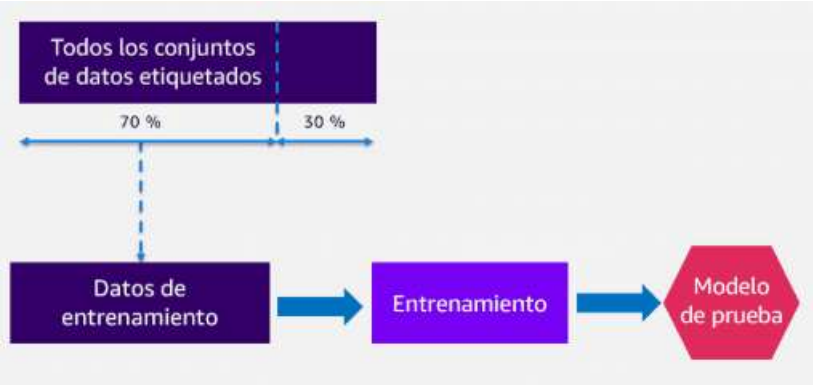
- **División de los datos de entrenamiento:**

  La creación de un modelo de ML requiere dividir los datos en dos conjuntos, como un *conjunto de entrenamiento* y un *conjunto de pruebas*. Divida los conjuntos con una proporción de 80:20 o 70:30.



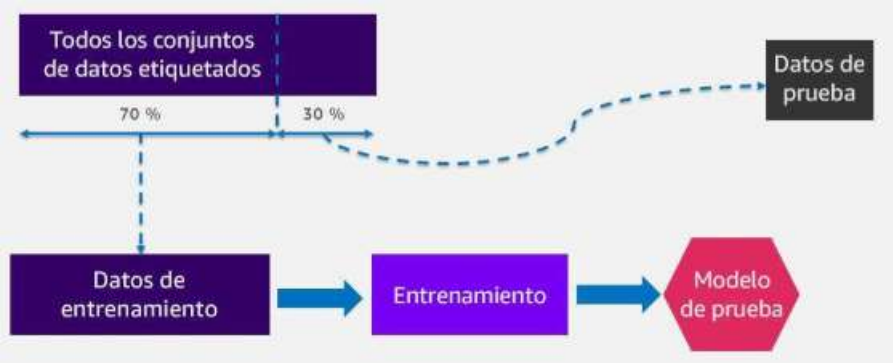
- **Entrenamiento con datos de entrenamiento:**

  Seleccione un algoritmo de entrenamiento que sea adecuado para resolver el problema empresarial. Cree un modelo con el algoritmo y los datos de entrenamiento como entradas. Repita el entrenamiento del modelo hasta que una métrica empresarial esperada, como la exactitud, alcance un valor aceptable.



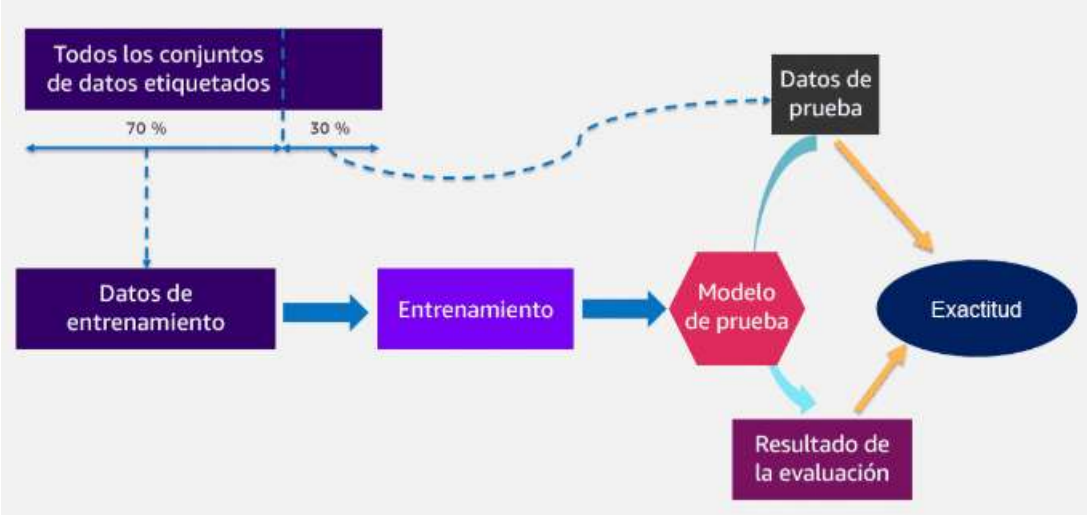
- **División de los datos de prueba:**

  El conjunto de datos de prueba que no se usó durante el entrenamiento del modelo se usa para medir el rendimiento del modelo.



- **Medición del rendimiento:**

  El modelo se prueba mediante la ejecución de inferencias con datos de prueba como entrada que producen métricas de evaluación. Puede evaluar las predicciones del modelo en función de la métrica objetivo (por ejemplo, la exactitud).



Tipos de algoritmo:

Los algoritmos de ML aceptan datos de entrenamiento como entrada para crear un modelo que se pueda usar para predecir resultados en datos no vistos que sean similares a los datos utilizados para entrenar el modelo. Por ejemplo, los algoritmos supervisados utilizan datos etiquetados que constan de características y etiquetas, mientras que los algoritmos no supervisados no requieren datos etiquetados.

La creación de modelos de ML con datos sin etiquetar requiere diferentes algoritmos de entrenamiento, y estos modelos abordan una clase diferente de problema empresarial. Por ejemplo, se puede usar un algoritmo de agrupación en clústeres para agrupar documentos por tema. Este algoritmo no requiere ejemplos etiquetados para crear el modelo.

El enfoque de este curso es el aprendizaje supervisado, por lo tanto, no se abordan los algoritmos de aprendizaje no supervisado.

El etiquetado de datos es el proceso de identificar datos sin procesar (imágenes, archivos de texto, videos, etc.) y añadir una o más etiquetas significativas e informativas para proporcionar contexto, de modo que un modelo de ML pueda aprender de él.

Para los datos etiquetados, se recomienda elegir un algoritmo adecuado. Estos son algunos ejemplos:



Algoritmos para datos etiquetados (supervisados).

- Utilice el algoritmo de regresión logística si el resultado a predecir es de naturaleza binaria (0/1). Por ejemplo, ¿la entrada es una imagen de un gato?
- Elija un clasificador de árbol de decisión, como bosque aleatorio o k vecinos más cercanos (KNN), si el resultado a predecir es de naturaleza multiclase (1/2/3/…). Por ejemplo, ¿cuál es la raza del gato en la imagen de entrada?
- Si el resultado que se va a predecir es de naturaleza continua, como cantidad, dinero, altura y peso, elija un algoritmo de regresión lineal. Algunos ejemplos de estos algoritmos son el regresor de árbol de decisión y el regresor de bosque aleatorio. Por ejemplo, ¿cuál es el número previsto de ventas de un producto?

EVALUACIÓN DE MODELOS.

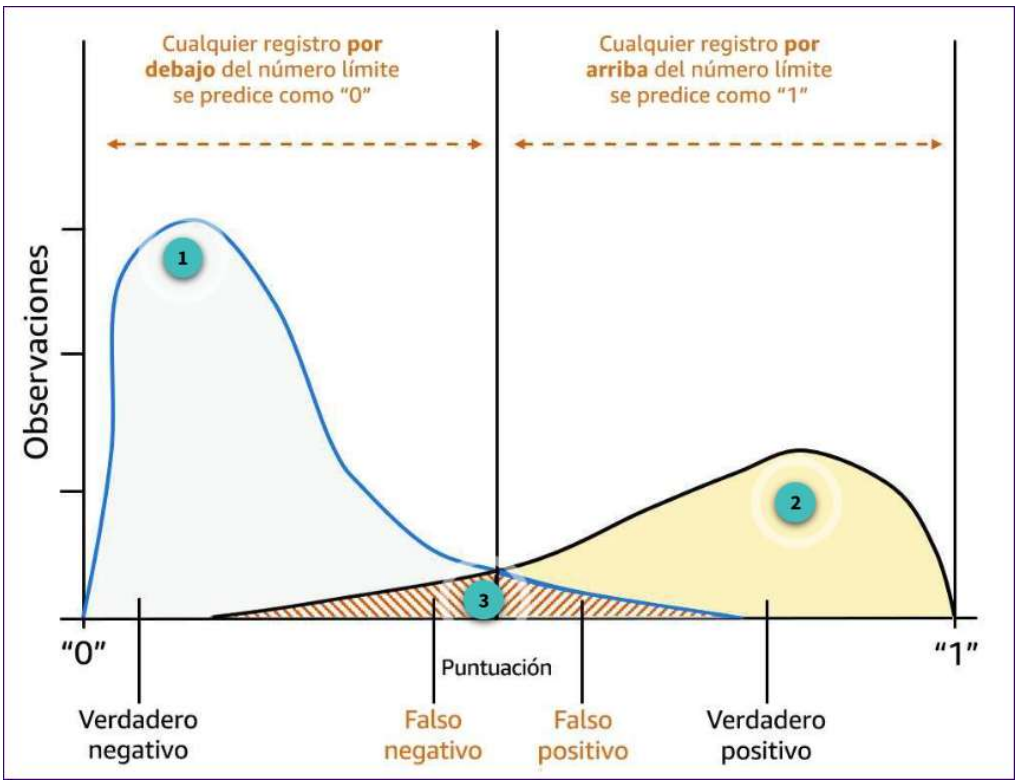
Exactitud del modelo.

En lugar de recordar únicamente los datos que se mostraron durante el entrenamiento, el modelo de ML busca descubrir patrones que se generalicen bien para datos desconocidos. Una vez que haya entrenado un modelo, es fundamental evaluar su rendimiento en muestras de datos que no ha visto antes y que no se utilizaron para entrenar el modelo. Ejecute el modelo para predecir la respuesta en el conjunto de datos de evaluación (prueba) que contuvo los datos retenidos. Para ello, compare la etiqueta prevista con la etiqueta real (ground truth).

En el ML se utilizan varias métricas para medir la exactitud predictiva de un modelo. La elección de la métrica de exactitud depende de la tarea de ML. Es importante revisar las métricas para decidir si el modelo funciona bien.

Clasificación binaria.

La *clasificación binaria* (un tipo de ML supervisado) es la tarea de clasificar los elementos de un conjunto en dos grupos en función de una regla de clasificación. Por ejemplo, estos dos grupos podrían ser una clase positiva y una clase negativa. El resultado real de muchos algoritmos de clasificación binaria es una puntuación de predicción. La puntuación indica la certeza del sistema de que la observación dada pertenece a la clase positiva.



1. **0 verdaderos:**

   Todos los 0 de respuesta verdaderos (conocidos) de sus datos de evaluación.
2. **1 verdadero:**

   Todos los 1 de respuesta verdaderos (conocidos) de sus datos de evaluación.
3. **Superposición:**

   Las áreas rayadas indican los registros para los que la respuesta se predijo incorrectamente en función del número límite seleccionado.

 Distribución de puntuaciones para un modelo de clasificación binaria: deberá decidir si la observación debe clasificarse como positiva o negativa. Para ello, interpretará la puntuación mediante la selección de un umbral (límite) de clasificación y la comparación de la puntuación con él. Cualquier observación con puntuaciones superiores al umbral se predicen como clase positiva. Las puntuaciones inferiores al umbral se predicen como clase negativa.

Las predicciones ahora se dividen en cuatro grupos según la respuesta conocida real y la respuesta prevista, de la siguiente manera:

- **Verdaderos positivos:** predicciones positivas correctas
- **Verdaderos negativos:** predicciones negativas correctas
- **Falsos positivos:** predicciones positivas incorrectas
- **Falsos negativos:** predicciones negativas incorrectas

Métricas de clasificación binaria:

Las métricas de exactitud de clasificación binaria cuantifican los dos tipos de predicciones correctas (verdaderos positivos y verdaderos negativos) y los dos tipos de errores (falsos positivos y falsos negativos). Cada métrica mide un aspecto diferente del modelo predictivo.

En las fórmulas de las métricas de evaluación se utilizan las siguientes variables.

- **TP** = número de verdaderos positivos
- **TN** = número de verdaderos negativos
- **FP** = número de falsos positivos
- **FN** = número de falsos negativos

Cada métrica mide un aspecto diferente del modelo predictivo.

- **Exactitud:**

  La **exactitud (ACC)** mide la fracción de predicciones correctas.

  Exactitud = (TP + TN)/(TP + TN + FP + FN)

$$\text{Exactitud} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precisión:**

  La **precisión** mide la fracción de positivos reales entre los ejemplos que se predicen como positivos.

  Precisión = (TP)/(TP + FP)

$$\text{Precisión} = \frac{TP}{TP + FP}$$

- **Exhaustividad:**

  La **exhaustividad** mide cuántos positivos reales se predijeron como positivos.

  Exhaustividad = (TP)/(TP + FN)

$$\text{Exhaustividad} = \frac{TP}{TP + FN}$$

- **Puntuación F1:**

  La **puntuación F1** es la media armónica de precisión y exhaustividad.

  Puntuación F1 = 2 x (exhaustividad x precisión)/(exhaustividad + precisión)

$$\text{Puntuación F1} = 2 \times \frac{\text{Exhaustividad} \times \text{Precisión}}{\text{Exhaustividad} + \text{Precisión}}$$

Matriz de confusión:

En el ML, una *matriz de confusión*, también denominada matriz de errores, es un diseño de tabla específico. Puede usarla para visualizar el rendimiento de un algoritmo, normalmente un algoritmo de clasificación de aprendizaje supervisado.

Clasificación binaria (1 o 0)	Predicción = 1	Predicción = 0
Real = 1	Verdadero positivo	Falso negativo
Real = 0	Falso positivo	Verdadero negativo

Un resultado positivo verdadero es el número total de veces que el modelo clasificó correctamente las etiquetas positivas del conjunto de datos reales. Un resultado falso negativo es el número total de clasificaciones incorrectas de etiquetas positivas como etiquetas negativas.

Del mismo modo, un resultado negativo verdadero es el número total de clasificaciones correctas de las etiquetas negativas. Un resultado falso positivo es el número total de veces que el modelo clasificó incorrectamente las etiquetas negativas como positivas.

Identificar la métrica correcta para un problema empresarial:

En función del problema empresarial, es posible que le interese más un modelo que funcione bien para un subconjunto específico de métricas del modelo. Por ejemplo, dos aplicaciones empresariales pueden tener requisitos muy diferentes para sus modelos de ML, de la siguiente manera:

- **Predicciones de alta precisión**

  Para una aplicación, se podría necesitar estar extremadamente seguro de que las predicciones positivas son realmente positivas (precisión alta). También es posible que se necesite estar dispuesto a aceptar que algunos ejemplos positivos se clasifiquen incorrectamente como negativos (exhaustividad moderada). Por ejemplo, se necesitaría una precisión alta para un filtro de spam.

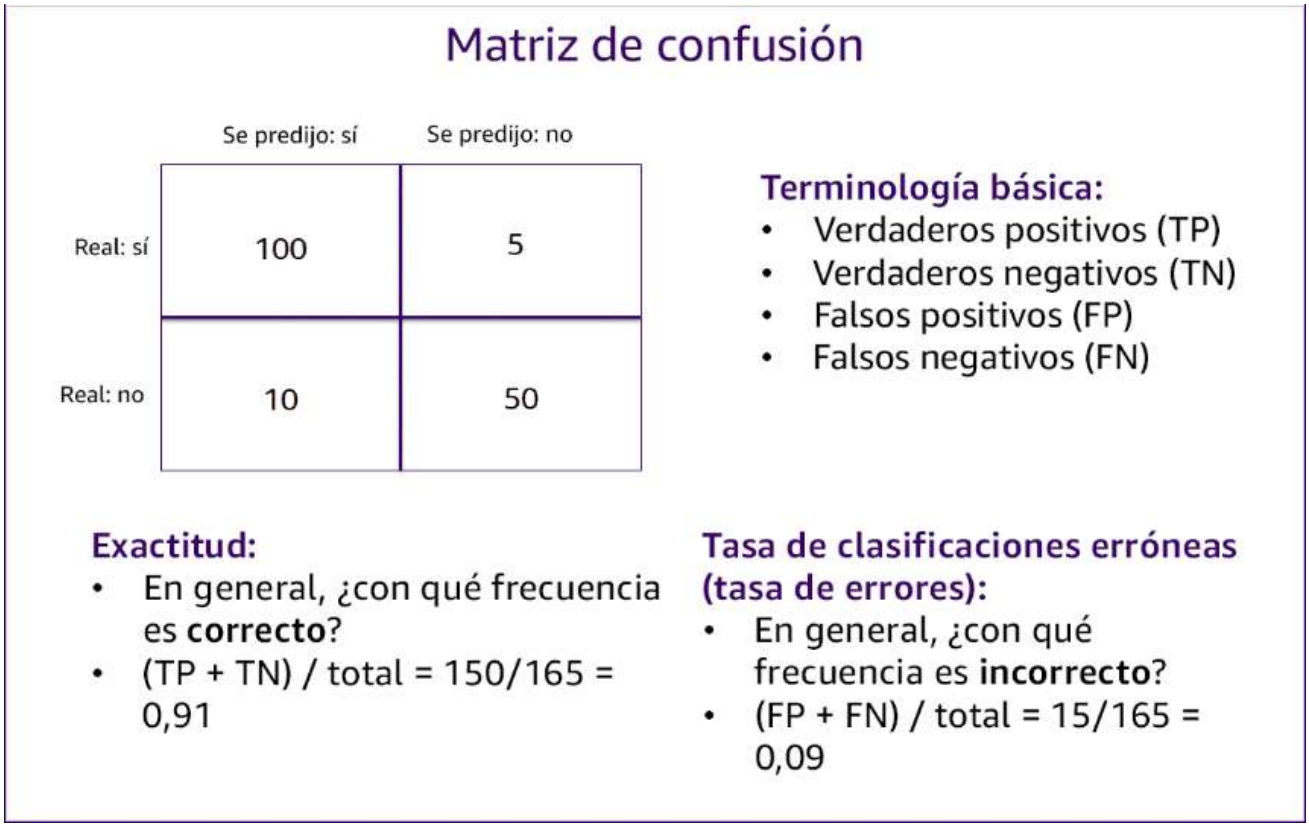
- **Predicciones de alta exactitud**

  Para otra aplicación, se podría necesitar predecir correctamente tantos ejemplos positivos como sea posible (exhaustividad alta) y aceptar que algunos ejemplos negativos se clasifiquen incorrectamente como positivos (precisión moderada). Por ejemplo, se querría una exactitud alta (con exhaustividad alta) para las pruebas de detección del cáncer.

Ejemplo de matriz de confusión:

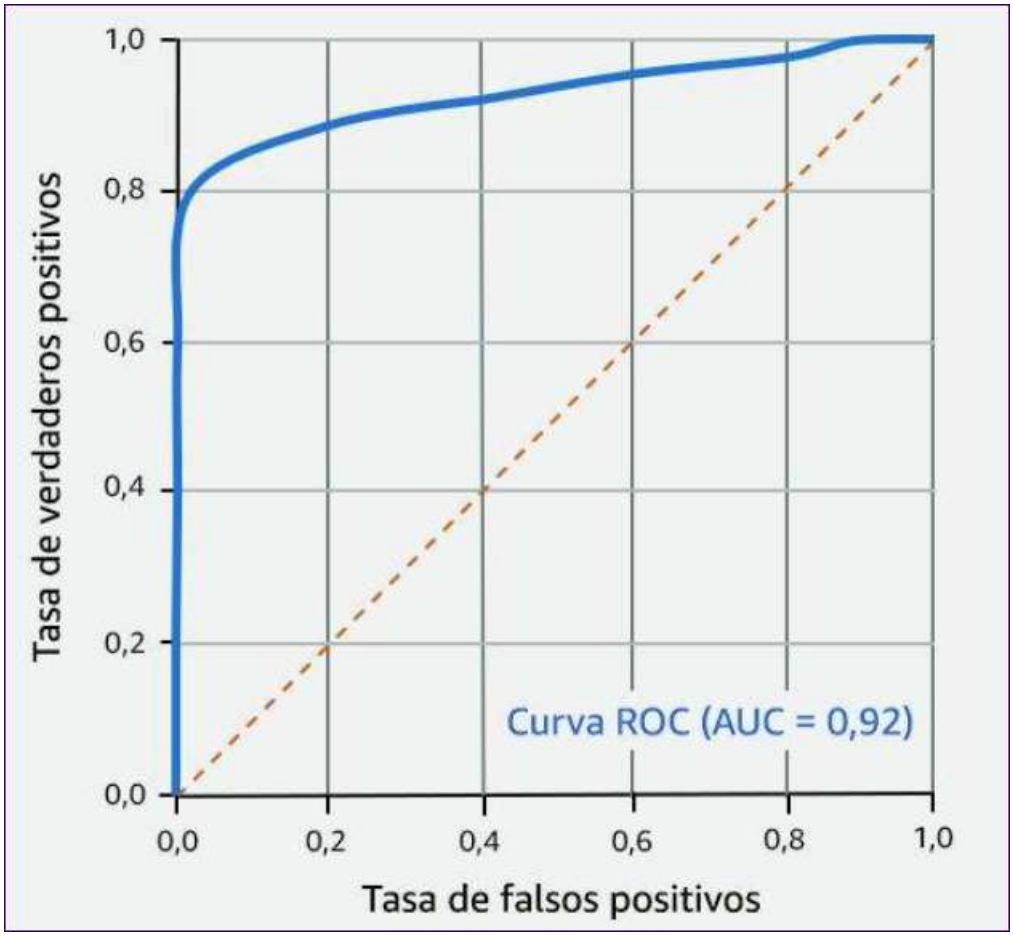
Supongamos que se diseña una nueva prueba para determinar si un paciente tiene un virus en particular. Para medir la exactitud de la prueba, esta se aplica a un número (165) de pacientes. Antes de hacerla, se sabe que 105 pacientes tienen el virus y 60 no lo tienen. Los resultados del experimento se presentan en la siguiente matriz

de confusión. El número de verdaderos positivos (TP) es 100 y el número de verdaderos negativos (TN) previsto es 50. Con la fórmula de exactitud, la prueba del virus es correcta en un 91 por ciento para este caso.



Curva ROC y AUC:

Una curva de la *característica operativa del receptor* (ROC) es un gráfico que muestra el rendimiento de un modelo de clasificación en todos los umbrales de clasificación. Esta curva traza dos parámetros: la *tasa de verdaderos positivos* y la *tasa de falsos positivos*. El *área bajo la curva* (AUC) mide el área bidimensional completa por debajo de toda la curva ROC.



El AUC mide la capacidad del modelo para predecir una puntuación más alta para los ejemplos positivos en comparación con los negativos. Como el AUC es independiente del umbral seleccionado, es posible hacerse una idea del rendimiento de predicción de un modelo a partir de la métrica AUC sin elegir un umbral.

Clasificación multiclase:

La *clasificación multiclase*, a menudo denominada clasificación multinomial, es el problema de clasificar los casos en una de tres o más clases en la clasificación estadística y el ML. A diferencia de los desafíos de la clasificación binaria, hacer predicciones no requiere seleccionar un umbral de puntuación. La clase (o etiqueta) con la puntuación prevista más alta es la respuesta prevista. A veces, es posible que solo se desee usar la respuesta proyectada si tiene una puntuación de predicción alta.

En esta situación, se puede decidir si se acepta la respuesta proyectada al establecer un umbral basado en las puntuaciones esperadas. Las métricas típicas utilizadas en el multiclase son las mismas que las métricas utilizadas en el caso de la clasificación binaria. La métrica se calcula para cada clase tratándola como un problema de clasificación binaria después de agrupar todas las demás clases como pertenecientes a la segunda.

Luego, la métrica binaria se promedia en todas las clases para obtener una métrica de promedio macro (que trata a cada clase por igual) o de promedio ponderado (ponderada por frecuencia de clase).

Modelos de regresión.

Los *modelos de regresión* se utilizan para predecir un valor continuo. Predecir los precios de una casa teniendo en cuenta sus características, como el tamaño, el precio, etc., es uno de los ejemplos más comunes de regresión. Es un tipo de ML supervisado. Para las tareas de regresión, las métricas de exactitud típicas son las siguientes:

- Raíz del error cuadrático medio (RMSE)
- Error absoluto medio porcentual (MAPE)

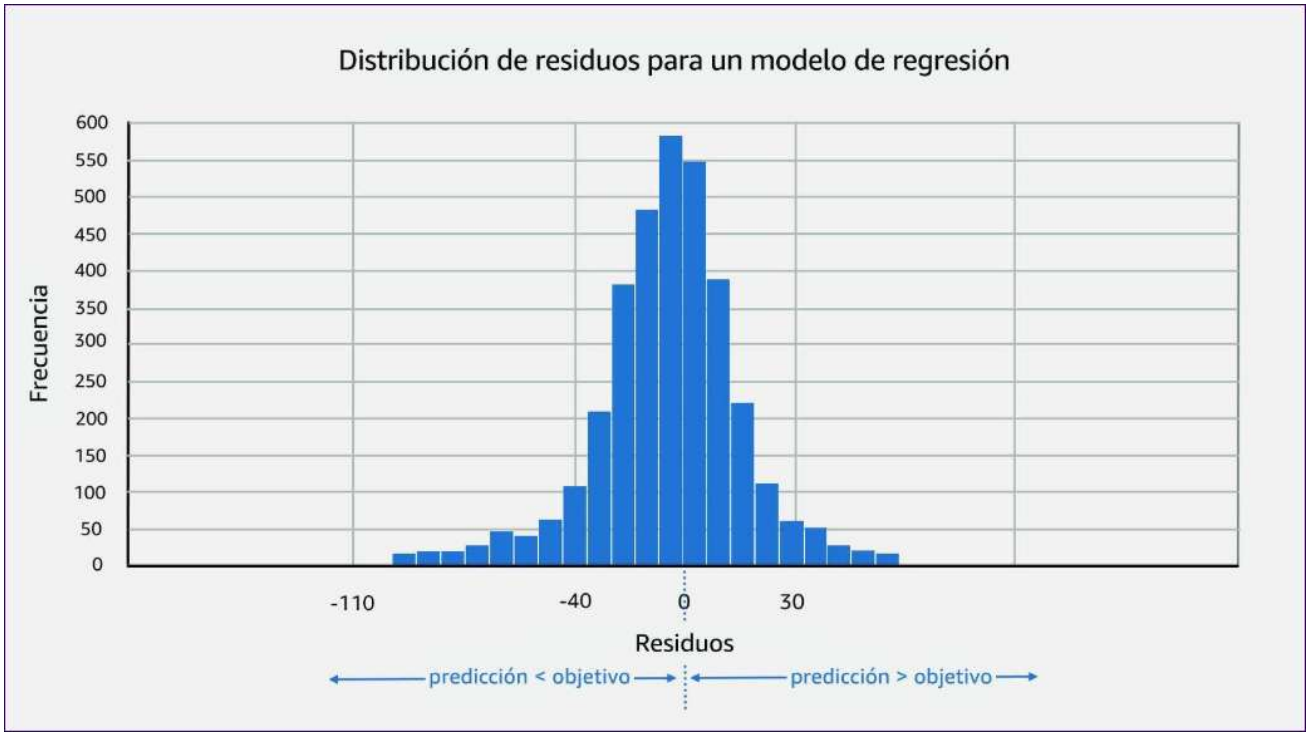
Estas métricas miden la distancia entre el objetivo numérico previsto y la respuesta numérica real (Ground Truth).

Residuos para modelos de regresión:

Es una práctica común revisar los residuos de los problemas de regresión. Un *residuo* de una observación en los datos de evaluación es la diferencia entre el objetivo verdadero y el objetivo previsto. Los residuos representan la parte del objetivo que el modelo no puede predecir. A continuación, se encuentran las pautas para los residuos:

- Un residuo positivo indica que el modelo está subestimando el objetivo. El objetivo real es mayor que el objetivo previsto.
- Un residuo negativo indica una sobreestimación. El objetivo real es menor que el objetivo previsto.

Cuando se distribuye en forma de campana y se centra en cero, el histograma de los residuos de los datos de evaluación indica que el modelo comete errores de forma aleatoria. No sobrepredice ni subpredice sistemáticamente ningún rango particular de valores objetivo. Si los residuos no generan una forma de campana centrada en cero, existe una estructura en el error de predicción del modelo. Agregar más variables al modelo puede ayudar a que este capture el patrón que no captura el modelo actual.



Un ejemplo de un histograma de residuos en los datos de evaluación distribuidos en forma de campana y centrados en cero.

MEJORAR EL RENDIMIENTO DEL MODELO.

Mejorar el rendimiento con machine learning.

La creación de un modelo de ML es un proceso iterativo. La obtención de un modelo que se adapte a sus necesidades normalmente implica repetir un proceso y probar algunas variaciones. Es posible que no obtenga un modelo altamente predictivo en la primera iteración o que desee optimizarlo para obtener predicciones aún mejores.

Para mejorar el rendimiento, puede iterar los siguientes pasos:



La mejora del rendimiento como proceso iterativo.

- **Recopilación de datos:** aumente la cantidad de ejemplos de entrenamiento.
- **Procesamiento de características:** añada más variables y un mejor procesamiento de características.
- **Ajuste de parámetros del modelo:** considere valores alternativos para los parámetros que utiliza el algoritmo de aprendizaje.

Desafíos relacionados con la recopilación y el procesamiento de datos.

Ahora que tiene una comprensión básica sobre el proceso de ML de alto nivel, aprenderá algunos puntos importantes que pueden ayudar durante la implementación.

Fuga de datos y cómo evitarla:

La fuga (o filtración) de datos ocurre cuando los datos de entrenamiento contienen información sobre el objetivo, pero no habrá datos similares disponibles cuando utilice el modelo para la predicción. Esto conduce a un alto rendimiento del conjunto de entrenamiento (y posiblemente incluso los datos de validación), pero un rendimiento deficiente en la producción del modelo.

 La fuga de características y la fuga por columnas se deben a la inclusión de una columna que tiene una etiqueta duplicada, que es un sustituto de la etiqueta o que es la propia etiqueta.

Sobreajuste:

El *sobreajuste* es un modelo de ML inaceptable que se produce cuando el modelo de ML proporciona predicciones exactas para los datos de entrenamiento, pero no para los datos nuevos. Cuando los científicos de datos utilizan modelos de ML para hacer predicciones, primero entrenan el modelo con un conjunto de datos conocido. Luego, en función de esta información, el modelo intenta predecir los resultados de los nuevos conjuntos de datos. **Un modelo sobreajustado puede ofrecer predicciones inexactas y no funcionar bien para todos los tipos de datos nuevos.**

Solo obtendrá predicciones exactas si el modelo de ML se generaliza para todos los tipos de datos de su dominio. El sobreajuste se produce cuando el modelo no puede generalizarse y, en cambio, se ajusta demasiado al conjunto de datos de entrenamiento. El sobreajuste sucede por diferentes motivos, como los se encuentran a continuación:

- El volumen de los datos de entrenamiento es demasiado pequeño y no contiene suficientes muestras de datos para representar con exactitud todos los valores de datos de entrada posibles.
- Los datos de entrenamiento contienen grandes volúmenes de información irrelevante, denominada datos *ruidosos*.
- El modelo se entrena durante demasiado tiempo con un único conjunto de datos de muestra.
- La complejidad del modelo es alta, por lo que aprende el ruido de los datos de entrenamiento.

Ejemplos de sobreajuste:

- **Análisis de fotos:**

  Considere un caso práctico en el que un modelo de ML debe analizar fotos e identificar cuáles contienen perros. El modelo se entrenó con un conjunto de datos que contenía una mayoría de fotos de perros en parques. El modelo podría aprender a usar el césped como característica para la clasificación. Por lo tanto, es posible que no reconozca a un perro dentro de una habitación.

- **Predicciones de éxito:**

  Un algoritmo de ML predice el rendimiento académico y el resultado de la graduación de un estudiante universitario mediante el análisis de varios factores. Estos factores pueden incluir los ingresos familiares del niño, el rendimiento académico anterior y las calificaciones académicas de los padres del niño. Sin embargo, los datos de la prueba solo incluyen candidatos de un género o grupo étnico específico. En este caso, el

sobreajuste hace que la exactitud de predicción del algoritmo disminuya para los candidatos cuyo género o etnia se encuentran fuera del conjunto de datos de prueba.

Evitar el sobreajuste:

Puede diversificar el conjunto de datos de entrenamiento y ajustar su escala para evitar el sobreajuste. También existen otras estrategias de ciencia de datos disponibles.

- **Detención temprana:**

  La *detención temprana* detiene la fase de entrenamiento antes de que el modelo de ML aprenda el ruido de los datos. Sin embargo, es importante elegir el momento adecuado; de lo contrario, el modelo seguirá sin dar resultados exactos.

- **Depuración:**

  Puede identificar varias características o parámetros que afectan la predicción final al momento de crear un modelo. La selección de características, o *depuración*, identifica las características más importantes del conjunto de entrenamiento y elimina las que son irrelevantes.

  Por ejemplo, para predecir si una imagen es de un animal o un ser humano, puede observar varios parámetros de entrada. El parámetro puede incluir la forma de la cara, la posición de las orejas, la estructura corporal, etc. Puede priorizar la forma de la cara e ignorar la forma de los ojos.

- **Regularización:**

  La *regularización* es un conjunto de técnicas de entrenamiento y optimización para reducir el sobreajuste. Estos métodos califican las características según su importancia para intentar eliminar los factores que no afectan los resultados de la predicción.

  Por ejemplo, los cálculos matemáticos aplican un valor de penalización a las entidades con un impacto mínimo. Pensemos en un modelo estadístico que intenta predecir los precios de las viviendas dentro de 20 años. La regularización daría un valor de penalización menor a características como el crecimiento demográfico y el ingreso medio anual. Daría un valor de penalización mayor a la temperatura media anual de la ciudad.

- **Ensamblaje:**

  El *ensamblaje* combina predicciones de varios algoritmos de ML independientes. Algunos modelos se denominan *aprendices débiles* porque sus resultados suelen ser inexactos. Los métodos de ensamblaje combinan todos los aprendices débiles para obtener resultados más exactos. Utilizan varios modelos para analizar los datos de las muestras y seleccionar los resultados más exactos.

  Los dos métodos principales de ensamblaje son *bagging* y *boosting*. En el método boosting, se entrenan diferentes modelos de ML uno tras otro para obtener el resultado final, mientras que en el bagging se los entrena en paralelo.

- **Aumento de datos:**

  El *aumento de datos* es una técnica de ML que cambia ligeramente los datos de muestra cada vez que el modelo los procesa. Esto se hace modificando los datos de entrada de manera sutil. Cuando se hace con moderación, el aumento de datos hace que los conjuntos de entrenamiento parezcan exclusivos del modelo e impide que el modelo aprenda sus características.

  Por ejemplo, el aumento de datos incluye la aplicación de transformaciones, como el desplazamiento, el volteo y la rotación, a las imágenes de entrada.

Subajuste y cómo evitarlo:

El *subajuste* es otro tipo de error que se produce cuando el modelo no puede determinar una relación significativa entre los datos de entrada y salida. Los modelos resultan *subajustados* si no se entrenaron durante el período de tiempo adecuado con una cantidad grande de puntos de datos.

Comparación entre subajuste y sobreajuste:

Los modelos subajustados presentan un nivel de sesgo alto. Pueden dar resultados inexactos tanto para los datos de entrenamiento como para el conjunto de pruebas. Sin embargo, los modelos sobreajustados presentan un nivel de varianza alto. Pueden dar resultados exactos para el conjunto de entrenamiento, pero no para el conjunto de prueba.

Un mayor entrenamiento del modelo disminuye el sesgo, pero la varianza podría aumentar. Cuando ajustan un modelo, los científicos de datos tienen como objetivo encontrar la posición óptima entre el subajuste y el sobreajuste. Un modelo bien ajustado puede establecer rápidamente la tendencia dominante para los conjuntos

de datos vistos y no vistos. Entrenar el modelo para más ciclos de entrenamiento y mezclar los datos después de cada ciclo puede ayudar a garantizar que la pérdida disminuya gradualmente a lo largo del entrenamiento. Un *ciclo de entrenamiento* es un flujo completo de datos de entrenamiento a través del algoritmo de ML.

Técnicas para gestionar datos desequilibrados:

Los *datos desequilibrados* se refieren a conjuntos de datos en los que la clase objetivo tiene una distribución desigual de las observaciones. Los datos desequilibrados pueden incluir una etiqueta de clase con un número muy alto de observaciones y otra etiqueta de clase con un número de observaciones muy bajo.

Por ejemplo, los clientes pueden tener un mínimo de datos etiquetados para las transacciones fraudulentas en relación con las transacciones no fraudulentas. El desequilibrio de clases generalmente es normal en los problemas de clasificación. Sin embargo, en algunos casos, este desequilibrio es grave porque la presencia de la clase mayoritaria es muy superior a la de la clase minoritaria.

- **Considerar las métricas de evaluación adecuadas:**
  Los clientes pueden considerar la puntuación F1, que mantiene el equilibrio entre precisión y exhaustividad. Mejora la puntuación solo si el clasificador identifica correctamente más elementos de una determinada clase. El AUC es otra métrica de evaluación alternativa.

- **Remuestro:**
  Para crear un conjunto de datos equilibrado a partir de uno desequilibrado, puede usar el *submuestreo* y el *sobremuestreo*. El submuestreo reduce el tamaño de la clase abundante para equilibrar el conjunto de datos. El sobremuestreo, por el contrario, se utiliza cuando la cantidad de datos es insuficiente. Aumenta el tamaño de las muestras poco frecuentes para intentar equilibrar el conjunto de datos.

  En lugar de descartar muestras abundantes, se generan nuevas muestras poco frecuentes mediante el uso de técnicas como la repetición, el proceso de arranque o la técnica de sobremuestreo sintético de minorías (SMOTE). SMOTE es un algoritmo que realiza el aumento de datos mediante la creación de puntos de datos sintéticos basados en los puntos de datos originales. SMOTE puede considerarse una versión avanzada del sobremuestreo o un algoritmo específico para el aumento de datos. La ventaja de SMOTE es que no se generan duplicados. En su lugar, se crean puntos de datos sintéticos que son ligeramente diferentes de los puntos de datos originales.

- **Validación cruzada k-fold:**

  La *validación cruzada* es una técnica que se utiliza para evaluar el rendimiento de los modelos de ML cuando hay una cantidad limitada de datos disponibles. El proceso implica dividir los datos en grupos o pliegues *k*. El modelo se entrena con pliegues *k-1* y luego se valida con el pliegue restante. Esto se repite *k* veces para que cada pliegue sirva como conjunto de validación una vez. A continuación, se promedia el rendimiento en todos los pliegues *k* para evaluar la habilidad general del modelo. Esta técnica ayuda a prevenir el sobreajuste y proporciona una estimación sólida del rendimiento del modelo.

  Este método garantiza que la puntuación de su modelo no dependa de la forma en que haya elegido el conjunto de entrenamiento y prueba. El conjunto de datos se divide en *k* subconjuntos y el método de retención se repite *k* veces.

- **Ensamblar diferentes conjuntos de datos con nuevas muestras:**

  La recopilación de más datos suele ser la forma más sencilla de mejorar la capacidad de generalización de un modelo. Sin embargo, los clasificadores comunes, como las regresiones logísticas o los bosques aleatorios, tienden a generalizar ignorando las clases poco frecuentes en los datos. Una práctica recomendada es entrenar varios modelos, cada uno con el conjunto completo de ejemplos de clases poco frecuentes y diferentes subconjuntos de la clase abundante.

  Por ejemplo, si el objetivo es un ensamblaje de 10 modelos, conserve los 1000 ejemplos de la clase poco frecuente y muestree al azar 10 000 ejemplos de la clase abundante. Divida los 10 000 ejemplos en 10 fragmentos y, a continuación, utilice cada fragmento para entrenar uno de los modelos. Esta técnica es fácil de implementar y se escala bien porque los modelos se pueden entrenar en paralelo en nodos de cómputo independientes. El ensamblaje de modelos también mejora la generalización. Por lo tanto, este método proporciona una forma eficiente de gestionar conjuntos de datos desequilibrados.

  Este método es eficiente y se escala horizontalmente sin problemas si tiene muchos datos, ya que solo puede entrenar y ejecutar sus modelos en diferentes nodos del clúster. Los modelos de ensamblaje también tienden a generalizar mejor, lo que hace que este método sea sencillo de manejar. Otra técnica consiste en utilizar diferentes proporciones de muestreo cuando se vuelve a muestrear los datos. El método anterior de ajustar la proporción entre las clases poco frecuentes y las abundantes puede funcionar bien. Sin embargo, la proporción óptima depende en gran medida de los datos y modelos específicos utilizados. En lugar de

entrenar a todos los modelos del ensamblaje con la misma proporción, podría ser beneficioso utilizar diferentes proporciones en todos los modelos.

Por ejemplo, con un ensamblaje de 10 modelos, un modelo puede usar una proporción de 1:1 entre las clases poco frecuentes y las abundantes. Otro modelo puede usar una proporción de 1:3 o incluso una proporción de 2:1. Utilizar diferentes proporciones de esta manera puede influir en la importancia relativa que el modelo otorga a cada clase durante el entrenamiento.

- **Consideraciones sobre el clasificador:**
  Un BalancedBaggingClassifier es lo mismo que un clasificador scikit-learn (sklearn) pero con equilibrio adicional. Incluye un paso adicional para equilibrar el conjunto de entrenamiento en el momento del ajuste para un muestrario determinado. El clasificador BalancedBaggingClassifier toma dos parámetros especiales: *sampling_strategy* y *replacement*. sampling_strategy decide el tipo de remuestreo requerido (por ejemplo, la *mayoría* remuestrea solo la clase mayoritaria, todos remuestrean todas las clases, etc.).

  Replacement decide si será una muestra con o sin reemplazo. Muchas veces, los clasificadores predicen realmente la probabilidad de pertenencia a una clase. Puede asignar las probabilidades de la predicción a una clase determinada en función de un umbral, que suele ser 0,5. Por ejemplo, si las probabilidades son inferiores a 0,5, pertenecen a una clase determinada. De lo contrario, pertenecen a la otra clase.

  Para problemas de clases desequilibradas, es posible que este umbral predeterminado no funcione correctamente. Debe cambiar el umbral al valor óptimo para que pueda separar las dos clases de manera eficiente. Puede usar curvas ROC y curvas de recuperación de precisión para encontrar el umbral óptimo para el clasificador. También puede utilizar un método de búsqueda en la red o buscar dentro de un conjunto de valores para identificar el valor óptimo.

- **Agrupar la clase abundante en clústeres:**
  Puede usar una técnica llamada agrupar a *clase abundante en clústeres*. En lugar de confiar en muestras aleatorias para cubrir la variedad de muestras de entrenamiento, agrupe la clase abundante en *r* clústeres, donde *r* es el número de casos en *r*. Para cada grupo, solo se conserva el *medoide* (el centro de un clúster). Luego, el modelo se entrena solo con la clase poco frecuente y con los medoides.

OPCIONES DE ENTRENAMIENTO DE MODELOS

Puede usar una de las siguientes tres opciones de entrenamiento de modelos en función de sus preferencias de control y flexibilidad.

- **Entrenamiento automatizado:**

  Los servicios de entrenamiento automatizado de modelos se encargan de todo el proceso por usted. Simplemente seleccione sus datos y el resultado deseado. El servicio lleva a cabo la ingeniería de características, la selección de algoritmos y el ajuste. Como resultado, la automatización permite iteraciones rápidas y posibilidades de exploración.



- **Entrenamiento guiado:**

  Las plataformas de entrenamiento de modelos guiado proporcionan sugerencias, pero le dan más control. Puede seleccionar algoritmos específicos y ajustar los parámetros del modelo y el entrenamiento. Esto ofrece un equilibrio entre la facilidad de uso y la personalización.



- **Entrenamiento de código personalizado:**

  Algunos servicios permiten agregar código personalizado para el entrenamiento avanzado o la integración de bibliotecas o marcos de trabajo externos. Esto requiere experiencia en codificación, pero otorga el mayor nivel de control y flexibilidad del entorno de ML.

